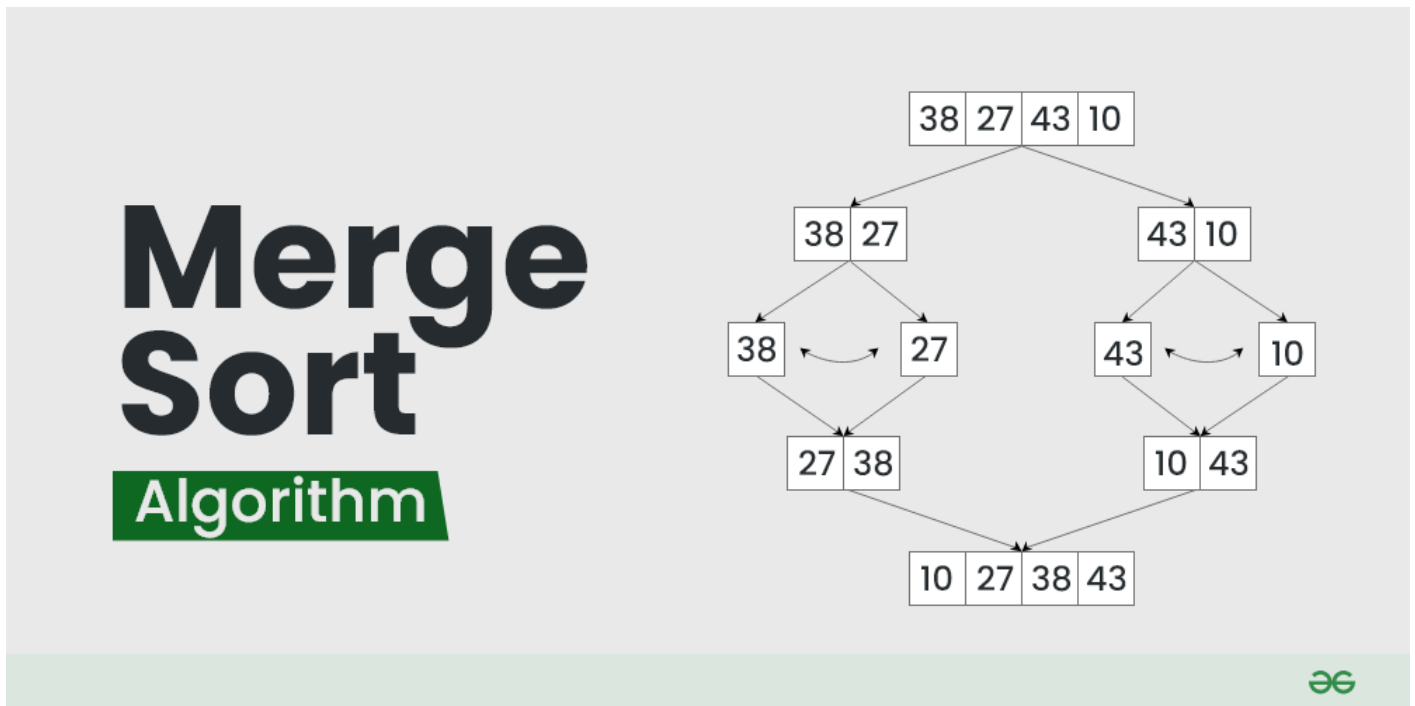


Merge Sort - Data Structure and Algorithms Tutorials

Last Updated : 30 Jul, 2025



Merge sort is a popular sorting algorithm known for its efficiency and stability. It follows the [divide-and-conquer](#) approach. It works by recursively dividing the input array into two halves, recursively sorting the two halves and finally merging them back together to obtain the sorted array.



Merge Sort Algorithm

How does Merge Sort work?

Here's a step-by-step explanation of how merge sort works:

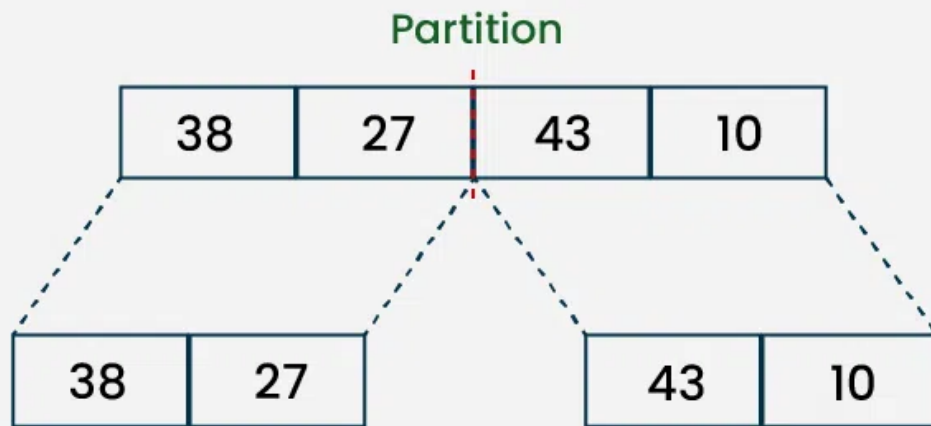
1. **Divide:** Divide the list or array recursively into two halves until it can no more be divided.
2. **Conquer:** Each subarray is sorted individually using the merge sort algorithm.
3. **Merge:** The sorted subarrays are **merged back together in sorted order**. The process continues until all elements from both subarrays have been merged.

Illustration of Merge Sort:

Let's sort the array or list [38, 27, 43, 10] using Merge Sort

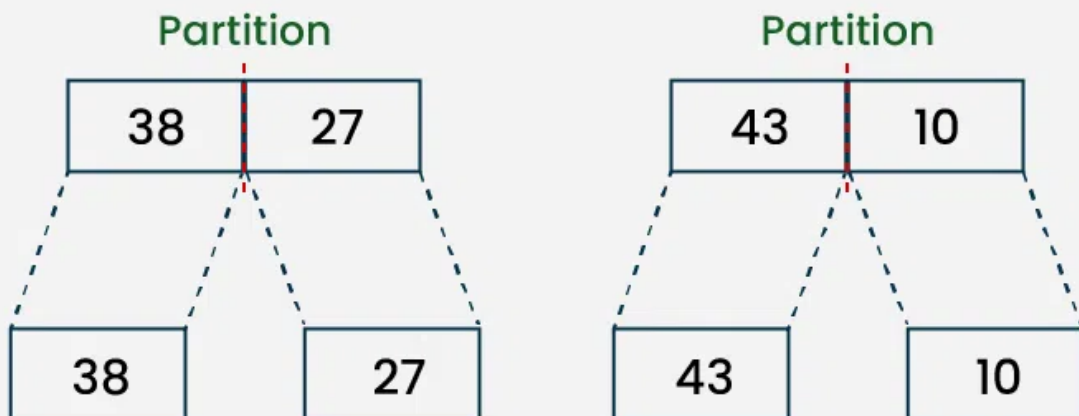
Step 1

Splitting the Array into two equal halves



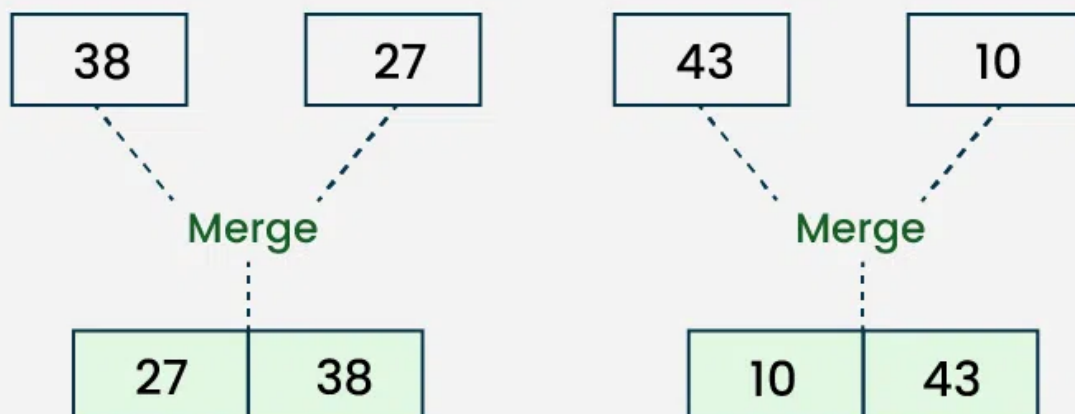
Step 2

Splitting the subarrays into two halves



Step 3

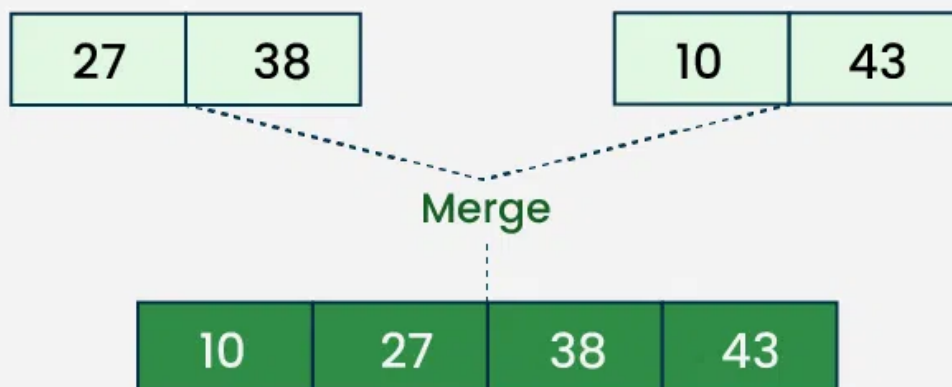
Merging unit length cells into sorted subarrays





Step 4

Merging sorted subarrays into the sorted array



Let's look at the working of above example:

Divide:

- *[38, 27, 43, 10]* is divided into *[38, 27]* and *[43, 10]* .
- *[38, 27]* is divided into *[38]* and *[27]* .
- *[43, 10]* is divided into *[43]* and *[10]* .

Conquer:

- *[38]* is already sorted.
- *[27]* is already sorted.
- *[43]* is already sorted.
- *[10]* is already sorted.

Merge:

- Merge *[38]* and *[27]* to get *[27, 38]* .
- Merge *[43]* and *[10]* to get *[10, 43]* .
- Merge *[27, 38]* and *[10, 43]* to get the final sorted list *[10, 27, 38, 43]*

Therefore, the sorted list is *[10, 27, 38, 43]* .

```
#include <bits/stdc++.h>
using namespace std;

// Merges two subarrays of arr[].
// First subarray is arr[left..mid]
// Second subarray is arr[mid+1..right]
void merge(vector<int>& arr, int left,
           int mid, int right){

    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Create temp vectors
    vector<int> L(n1), R(n2);

    // Copy data to temp vectors L[] and R[]
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0;
    int k = left;

    // Merge the temp vectors back
    // into arr[left..right]
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        }
        else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // Copy the remaining elements of L[],
    // if there are any
```

```

while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
}

// Copy the remaining elements of R[],
// if there are any
while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}
}

// begin is for left index and end is right index
// of the sub-array of arr to be sorted
void mergeSort(vector<int>& arr, int left, int right){

    if (left >= right)
        return;

    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

// Driver code
int main(){

    vector<int> arr = {38, 27, 43, 10};
    int n = arr.size();

    mergeSort(arr, 0, n - 1);
    for (int i = 0; i < arr.size(); i++)
        cout << arr[i] << " ";
    cout << endl;

    return 0;
}

```

Output

10 27 38 43

Recurrence Relation of Merge Sort

The recurrence relation of merge sort is:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & \text{if } n > 1 \end{cases}$$

- $T(n)$ Represents the total time taken by the algorithm to sort an array of size n .
- $2T(n/2)$ represents time taken by the algorithm to recursively sort the two halves of the array. Since each half has $n/2$ elements, we have two recursive calls with input size as $(n/2)$.
- $O(n)$ represents the time taken to merge the two sorted halves

Complexity Analysis of Merge Sort

- **Time Complexity:**
 - **Best Case:** $O(n \log n)$, When the array is already sorted or nearly sorted.
 - **Average Case:** $O(n \log n)$, When the array is randomly ordered.
 - **Worst Case:** $O(n \log n)$, When the array is sorted in reverse order.
- **Auxiliary Space:** $O(n)$, Additional space is required for the temporary array used during merging.

Applications of Merge Sort:

- Sorting large datasets
- [External sorting](#) (when the dataset is too large to fit in memory)
- [Inversion counting](#)
- Merge Sort and its variations are used in library methods of programming languages.

- Its variation [TimSort](#) is used in Python, Java Android and Swift. The main reason why it is preferred to sort non-primitive types is stability which is not there in QuickSort.
- [Arrays.sort in Java](#) uses QuickSort while [Collections.sort](#) uses MergeSort.
- It is a preferred algorithm for sorting Linked lists.
- It can be easily parallelized as we can independently sort subarrays and then merge.
- The merge function of merge sort to efficiently solve the problems like [union and intersection of two sorted arrays](#).

Advantages and Disadvantages of Merge Sort

Advantages

- **Stability** : Merge sort is a stable sorting algorithm, which means it maintains the relative order of equal elements in the input array.
- **Guaranteed worst-case performance:** Merge sort has a worst-case time complexity of $O(N \log N)$, which means it performs well even on large datasets.
- **Simple to implement:** The divide-and-conquer approach is straightforward.
- **Naturally Parallel** : We independently merge subarrays that makes it suitable for parallel processing.

Disadvantages

- **Space complexity:** Merge sort requires additional memory to store the merged sub-arrays during the sorting process.
- **Not in-place:** Merge sort is not an in-place sorting algorithm, which means it requires additional memory to store the sorted data. This can be a disadvantage in applications where memory usage is a concern.
- Merge Sort is **Slower than QuickSort in general** as QuickSort is more cache friendly because it works in-place.

Quick Links:

- [Merge Sort Based Coding Questions](#)
- [Bottom up \(or Iterative\) Merge Sort](#)

- [Recent Articles on Merge Sort](#)
- [Top Sorting Interview Questions and Problems](#)
- [Practice problems on Sorting algorithm](#)
- [Quiz on Merge Sort](#)

Merge Sort Introduction

[Visit Course ↗](#)[Comment](#)[More info ▼](#)[Advertise with us](#)[Next Article >](#)[Analysis of Algorithms](#)

Similar Reads

Basics & Prerequisites



Logic Building Problems

Logic building is about creating clear, step-by-step methods to solve problems using simple rules and principles. It's the heart of coding, enabling programmers to think, reason, and arrive at smart...

🕒 2 min read

Analysis of Algorithms

Analysis of Algorithms is a fundamental aspect of computer science that involves evaluating performance of algorithms and programs. Efficiency is measured in terms of time and space. Basics on...

🕒 1 min read

Data Structures



Algorithms



Advanced



Interview Preparation



Practice Problem



